

Primer Parcial Práctico

Cinemática Directa e Inversa del Robot KUKA KR 7 R900-3 con ROS 2 Jazzy

Grupo 7

Yohel Felipe Tancara Poma

Jhamil Alexis Gutierrez Huarayo

IMT-342 Robótica – Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—

INSTRUCCIÓN: Redactar entre 120 y 180 palabras. Resumir el robot asignado, la obtención de los parámetros Denavit–Hartenberg, el desarrollo de la cinemática directa, el método utilizado para la cinemática inversa, la implementación en ROS 2 y los principales resultados de validación. No incluir citas ni resultados que no aparezcan desarrollados posteriormente.

[REEMPLAZAR POR EL RESUMEN DEL GRUPO.]

Index Terms—Denavit–Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, manipulador serial.

I. REQUISITOS MÍNIMOS DE LA ENTREGA

INSTRUCCIÓN: Esta sección funciona como lista de control. Puede mantenerse en el borrador y eliminarse en la versión final. El informe debe demostrar desarrollo matemático propio y validación en ROS 2.

La entrega del Primer Parcial Práctico debe contener, como mínimo:

1. Identificación completa del robot asignado y de su cadena cinemática.
2. Asignación justificada de marcos de referencia.
3. Tabla completa de parámetros Denavit–Hartenberg.
4. Matriz homogénea general y matrices particulares de cada transformación.
5. Cinemática directa simbólica y/o semisimbólica.
6. Obtención de la posición y orientación del efector final.
7. Desarrollo del Jacobiano posicional empleado en la cinemática inversa.
8. Formulación e implementación de la cinemática inversa de posición para un objetivo cartesiano (x, y, z) .
9. Recepción del objetivo mediante el tópico `/target` con mensajes `geometry_msgs/msg/Point`.
10. Publicación de la solución articular mediante `/joint_states`.
11. Criterio de inicialización, convergencia, tolerancia y límites articulares.
12. Validación matemática mediante al menos tres configuraciones articulares.
13. Validación de cinemática inversa mediante al menos tres objetivos alcanzables.
14. Implementación propia en ROS 2.

15. Evidencia en RViz2 y Joint State Publisher GUI.
16. Estructura del workspace y comandos de compilación/ejecución.
17. Enlace a un repositorio GitHub reproducible.
18. README del repositorio con instrucciones completas desde una computadora nueva.
19. Conclusiones técnicas del grupo.

II. DESCRIPCIÓN DEL ROBOT ASIGNADO

INSTRUCCIÓN: Describir únicamente el robot asignado al grupo. Incluir fabricante, modelo, número de grados de libertad utilizados, tipo de articulaciones, alcance aproximado si la fuente oficial lo proporciona y la identificación del marco base y del efector final. Citar la documentación oficial utilizada.

II-A. Datos generales

El robot asignado al Grupo 7 es el KUKA KR 7 R900-3, perteneciente a la familia de robots industriales de pequeño tamaño de KUKA. El manipulador dispone de seis grados de libertad, implementados mediante seis articulaciones revolutas. De acuerdo con la documentación del fabricante, posee un alcance máximo aproximado de 902 mm y una capacidad de carga nominal de 7 kg [1].

En el modelo utilizado en ROS 2, la cadena cinemática comienza en el marco `base_link` y se consideran las articulaciones `joint_1` hasta `joint_6`. Como marco asociado al efector final se utiliza `tool0`. Las magnitudes lineales se expresan en metros y las variables articulares en radianes.

Cuadro I
DATOS PRINCIPALES DEL ROBOT

Dato	Valor
Fabricante	KUKA
Modelo	KR 7 R900-3
Grados de libertad	6
Tipo de articulaciones	6 revolutas
Alcance máximo	0.902 m
Carga nominal	7 kg
Marco base	<code>base_link</code>
Efector final	<code>tool0</code>
Unidades empleadas	SI: metros y radianes

II-B. Cadena cinemática

El KUKA KR 7 R900-3 presenta una cadena cinemática serial compuesta por seis articulaciones revolutas. Cada articulación conecta de manera sucesiva los eslabones del manipulador, desde el marco fijo `base_link` hasta el efector final `tool0`.

La cadena utilizada en el modelo ROS 2 es:

```
base_link  $\xrightarrow{q_1}$  link_1  $\xrightarrow{q_2}$  link_2  $\xrightarrow{q_3}$  link_3
link_3  $\xrightarrow{q_4}$  link_4  $\xrightarrow{q_5}$  link_5  $\xrightarrow{q_6}$  link_6
link_6  $\rightarrow$  tool0.
```

INSTRUCCIÓN: Insertar una figura del robot donde se distingan claramente las articulaciones $q_1, \dots, q_n$. La imagen puede provenir de la documentación oficial o ser una captura propia de RViz2, pero debe citarse o identificarse correctamente.

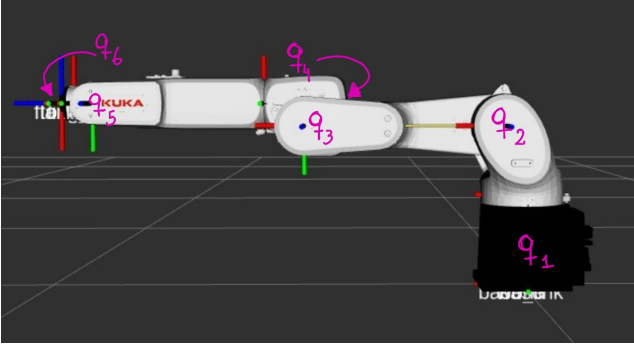


Figura 1. Robot KUKA KR 7 R900-3 y articulaciones consideradas.

III. ASIGNACIÓN DE MARCOS DE REFERENCIA

Para el desarrollo cinemático se definieron marcos de referencia siguiendo la convención Denavit–Hartenberg estándar. Para cada articulación revoluta, el eje z_{i-1} se alineó con su eje de rotación. El eje x_i se seleccionó sobre la normal común entre dos ejes z consecutivos, mientras que el eje y_i se obtuvo aplicando la regla de la mano derecha.

Los marcos definidos mediante este procedimiento no coinciden necesariamente con los marcos propios de los eslabones del URDF. Por este motivo, el modelo URDF/XACRO se utiliza como referencia geométrica y posteriormente como medio de validación del modelo cinemático desarrollado.

INSTRUCCIÓN: Esta parte es obligatoria. Mostrar cómo se colocó cada marco antes de presentar la tabla DH. El eje z_{i-1} debe asociarse con el eje de movimiento de la articulación correspondiente en la convención DH estándar. Explicar la elección de x_i a partir de la normal común entre ejes consecutivos. El eje y_i se obtiene mediante la regla de la mano derecha. Si se requieren marcos intermedios fijos, deben mostrarse y justificarse.

Para cada marco se debe indicar:

- origen O_i ;
- orientación de x_i , y_i y z_i ;
- articulación asociada;
- relación geométrica con el marco anterior;

- cualquier desplazamiento fijo o corrección angular requerida por el modelo.

La asignación de marcos se realizó siguiendo la geometría de los ejes articulares observada en el modelo URDF. Los marcos se definieron de la siguiente manera:

- **Marco {0}**: su origen O_0 se ubica sobre el eje de la primera articulación, en la base del robot. El eje z_0 coincide con el eje de rotación de q_1 .
- **Marco {1}**: su origen O_1 se localiza sobre el eje de la segunda articulación. El eje z_1 coincide con el eje de rotación de q_2 y x_1 se orienta sobre la normal común entre los ejes de q_1 y q_2 .
- **Marco {2}**: el origen O_2 se sitúa sobre el eje de q_3 . Debido a que los ejes de q_2 y q_3 son paralelos, x_2 se escoge sobre la normal común entre ambos ejes y z_2 coincide con el eje de q_3 .
- **Marco {3}**: su origen O_3 se establece sobre el eje de la cuarta articulación, en el punto alcanzado por la normal común entre los ejes de q_3 y q_4 . El eje z_3 coincide con el eje de rotación de q_4 .
- **Marco {4}**: se coloca en la intersección de los ejes de q_4 y q_5 . El eje z_4 coincide con el eje de rotación de q_5 .
- **Marco {5}**: se coloca en la intersección de los ejes de q_5 y q_6 . El eje z_5 coincide con el eje de rotación de q_6 .
- **Marco {6}**: corresponde al extremo de la cadena cinemática. La transformación fija entre el último eslabón y `tool0` se conserva de forma separada para representar correctamente el efector final.

En todos los casos, el eje y_i se obtiene mediante la regla de la mano derecha una vez definidos x_i y z_i .

INSERTAR FIGURA PROPIA
DEL ROBOT CON TODOS LOS FRAMES

Figura 2. Asignación de marcos de referencia del robot KUKA KR 7 R900-3.

III-A. Convención utilizada

En este trabajo se utilizará la convención Denavit–Hartenberg estándar para modelar la cinemática del robot KUKA KR 7 R900-3. Bajo esta convención, cada transformación homogénea entre marcos consecutivos se expresa mediante cuatro parámetros: θ_i , d_i , a_i y α_i . La matriz homogénea elemental empleada es la correspondiente a la formulación estándar de Denavit–Hartenberg.

$$A_{i-1}^i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i). \quad (1)$$

Su forma matricial es:

$$A_{i-1}^i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

donde $c_{\theta_i} = \cos \theta_i$ y $s_{\theta_i} = \sin \theta_i$.

INSTRUCCIÓN: Si el grupo decide utilizar DH modificado, debe cambiar explícitamente la ecuación, explicar la convención y mantenerla durante todo el informe. No se permite mezclar DH estándar y modificado sin justificación.

IV. PARÁMETROS DENAVIT–HARTENBERG

INSTRUCCIÓN: Construir la tabla a partir de la geometría real del robot y del modelo URDF/XACRO. No copiar tablas de Internet sin verificar su correspondencia con los frames usados por el grupo. Todas las distancias deben expresarse en metros y los ángulos en radianes. Incluir offsets constantes dentro de θ_i cuando corresponda.

IV-A. Verificación geométrica de los parámetros

INSTRUCCIÓN: Explicar de dónde se obtuvo cada distancia y cada offset angular. Comparar, cuando sea posible, con dimensiones del URDF/XACRO o documentación oficial. Si existen transformaciones fijas adicionales, incluirlas como matrices separadas en lugar de ocultarlas.

V. CINEMÁTICA DIRECTA

La cinemática directa se obtiene mediante:

$$T_0^n = A_0^1 A_1^2 \cdots A_{n-1}^n. \quad (3)$$

Para un manipulador de seis grados de libertad:

$$T_0^6 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6. \quad (4)$$

La transformación final tiene la forma:

$$T_0^6 = \begin{bmatrix} R_0^6 & p_0^6 \\ 0 & 1 \end{bmatrix}, \quad (5)$$

donde:

$$p_0^6 = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad R_0^6 \in SO(3). \quad (6)$$

V-A. Matrices homogéneas individuales

INSTRUCCIÓN: Escribir explícitamente todas las matrices A_{i-1}^i después de sustituir los parámetros DH. Deben conservarse las variables articulares. No basta con pegar la matriz genérica. Si son demasiado extensas, pueden pasar expresiones auxiliares a un apéndice, pero las matrices fundamentales deben aparecer en el informe.

$$A_0^1 = \begin{bmatrix} \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

$$A_1^2 = \begin{bmatrix} \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

INSTRUCCIÓN: Continuar hasta la última transformación.

V-B. Matriz de transformación total

INSTRUCCIÓN: Presentar T_0^6 de manera simbólica o semisimbólica. Si la expansión completa es excesivamente larga, definir expresiones auxiliares, por ejemplo $c_{23} = \cos(q_2 + q_3)$, e indicar claramente cómo se obtuvo.

$$T_0^6 = [\text{RESULTADO DEL PRODUCTO MATRICIAL}]. \quad (9)$$

V-C. Posición y orientación del efector

Extraer:

$$p(q) = \begin{bmatrix} p_x(q) \\ p_y(q) \\ p_z(q) \end{bmatrix}. \quad (10)$$

INSTRUCCIÓN: Mostrar las expresiones finales de p_x , p_y y p_z . Para la orientación, presentar la matriz de rotación R_0^6 y, adicionalmente, una representación interpretable: cuaternión o ángulos de Euler. Indicar la convención de Euler utilizada si corresponde.

V-D. Comprobaciones matemáticas

La matriz de rotación debe cumplir aproximadamente:

$$(R_0^6)^T R_0^6 \approx I, \quad (11)$$

$$\det(R_0^6) \approx 1. \quad (12)$$

INSTRUCCIÓN: Verificar estas propiedades al menos para una configuración numérica y mostrar el resultado. También comprobar que la última fila de toda transformación homogénea sea $[0 \ 0 \ 0 \ 1]$.

VI. JACOBIANO DEL MANIPULADOR

INSTRUCCIÓN: Para este parcial, el Jacobiano se utilizará principalmente para resolver la cinemática inversa de **posición**. El grupo debe obtenerlo a partir de su propio modelo cinemático. No basta con llamar una función externa sin explicar su formulación.

Para una articulación revoluta, la contribución lineal puede obtenerse mediante:

$$J_{v_i} = z_{i-1} \times (o_n - o_{i-1}), \quad (13)$$

donde z_{i-1} es el eje de la articulación expresado en el marco base, o_{i-1} es el origen del marco correspondiente y o_n la posición del efector final.

Por tanto, para un manipulador de n articulaciones se empleará el Jacobiano posicional:

Cuadro II
PARÁMETROS DENAVIT–HARTENBERG DEL ROBOT **[MODELO]**

Transformación	θ_i	d_i [m]	a_i [m]	α_i [rad]	Descripción
A_0^1	$[q_1 + \dots]$	$[...]$	$[...]$	$[...]$	base $\rightarrow$ link 1
A_1^2	$[q_2 + \dots]$	$[...]$	$[...]$	$[...]$	link 1 $\rightarrow$ link 2
A_2^3	$[q_3 + \dots]$	$[...]$	$[...]$	$[...]$	link 2 $\rightarrow$ link 3
A_3^4	$[q_4 + \dots]$	$[...]$	$[...]$	$[...]$	link 3 $\rightarrow$ link 4
A_4^5	$[q_5 + \dots]$	$[...]$	$[...]$	$[...]$	link 4 $\rightarrow$ link 5
A_5^6	$[q_6 + \dots]$	$[...]$	$[...]$	$[...]$	link 5 $\rightarrow$ efector

$$J_v(q) = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} & \dots & \frac{\partial p_x}{\partial q_n} \\ \frac{\partial p_y}{\partial q_1} & \dots & \frac{\partial p_y}{\partial q_n} \\ \frac{\partial p_z}{\partial q_1} & \dots & \frac{\partial p_z}{\partial q_n} \end{bmatrix}. \quad (14)$$

VI-A. Jacobiano utilizado en el proyecto

INSTRUCCIÓN: Presentar el Jacobiano posicional utilizado por el algoritmo de IK. Puede obtenerse analíticamente a partir de $p(q)$, geoméricamente a partir de los frames o simbólicamente con SymPy, pero el informe debe mostrar de dónde sale.

$$J_v(q) = \text{[INSERTAR JACOBIANO POSICIONAL]}. \quad (15)$$

VI-B. Consideración de singularidades

INSTRUCCIÓN: Explicar brevemente qué ocurre cuando el Jacobiano pierde rango o se encuentra cerca de una singularidad y cómo afecta esto al método iterativo. Para este primer parcial no se exige un estudio exhaustivo de singularidades.

VII. CINEMÁTICA INVERSA

VII-A. Definición del problema

Para este Primer Parcial Práctico, la cinemática inversa tendrá como objetivo obligatorio alcanzar una **posición cartesiana** deseada del efector final:

$$p_d = \begin{bmatrix} x_d \\ y_d \\ z_d \end{bmatrix}. \quad (16)$$

El algoritmo debe encontrar una configuración articular:

$$q^* = [q_1 \quad q_2 \quad \dots \quad q_n]^T \quad (17)$$

tal que:

$$p(q^*) \approx p_d. \quad (18)$$

INSTRUCCIÓN: No se exige resolver orientación en la IK de este parcial. La orientación sí debe obtenerse en la cinemática directa a partir de T_0^m . Si algún grupo decide resolver pose completa como ampliación, debe documentar adicionalmente el error de orientación y el Jacobiano correspondiente.

VII-B. Objetivo recibido mediante ROS 2

El punto objetivo debe recibirse mediante el tópic:

/target

utilizando el mensaje:

geometry_msgs/msg/Point.

Una prueba debe poder enviarse desde terminal mediante un comando equivalente a:

```
ros2 topic pub /target geometry_msgs/msg/Point \
" {x:_0.40, _y:_0.20, _z:_0.50}" --once
```

INSTRUCCIÓN: Los valores del ejemplo son únicamente ilustrativos. Cada grupo debe escoger objetivos alcanzables para su robot y documentar los valores utilizados en sus pruebas.

VII-C. Método numérico iterativo

A partir de la cinemática directa, se define el error de posición en la iteración k :

$$e_k = p_d - p(q_k). \quad (19)$$

El Jacobiano posicional se evalúa en la configuración actual:

$$J_k = J_v(q_k). \quad (20)$$

La actualización articular puede realizarse mediante la pseudoinversa:

$$q_{k+1} = q_k + \alpha J_k^+ e_k, \quad (21)$$

donde α es el factor de actualización y J_k^+ es la pseudoinversa del Jacobiano.

Cuando corresponda y sea matemáticamente válida, puede expresarse como:

$$J^+ = J^T (J J^T)^{-1}. \quad (22)$$

INSTRUCCIÓN: Si se utiliza `numpy.linalg.pinv`, debe explicarse que corresponde al cálculo numérico de la pseudoinversa. No se exige implementar Damped Least Squares en este primer parcial; puede utilizarse únicamente como ampliación si el grupo desea estudiar comportamiento cercano a singularidades.

VII-D. Configuración inicial

El algoritmo debe partir de una configuración:

$$q_0 = [q_{1,0} \quad q_{2,0} \quad \cdots \quad q_{n,0}]^T. \quad (23)$$

INSTRUCCIÓN: Documentar los valores iniciales utilizados. Deben corresponder a una configuración válida del robot y respetar sus límites articulares. Puede utilizarse una configuración home, el estado actual del robot o una semilla razonable elegida por el grupo.

VII-E. Articulaciones utilizadas

INSTRUCCIÓN: El grupo debe indicar qué articulaciones intervienen en la solución numérica. Si alguna articulación se mantiene fija durante la IK, debe indicarse su valor y justificarse. No asumir que todos los robots deben fijar las mismas articulaciones.

VII-F. Límites articulares

INSTRUCCIÓN: Presentar los límites de cada articulación utilizados por el algoritmo. Los valores deben provenir del URDF/XACRO o de documentación oficial. La solución final debe respetarlos.

Cuadro III
LÍMITES ARTICULARES UTILIZADOS

Joint	Mínimo [rad]	Máximo [rad]
q_1	[...]	[...]
q_2	[...]	[...]
q_3	[...]	[...]
q_4	[...]	[...]
q_5	[...]	[...]
q_6	[...]	[...]

VII-G. Criterio de convergencia

El error euclidiano de posición se define como:

$$e_p = \|p_d - p(q_k)\|_2. \quad (24)$$

El algoritmo se considera convergente cuando:

$$e_p < \varepsilon \quad (25)$$

antes de alcanzar el número máximo de iteraciones.

INSTRUCCIÓN: El grupo debe indicar el valor de ε , el factor α , el máximo de iteraciones y cualquier criterio adicional utilizado.

VII-H. Publicación de la solución articular

Una vez obtenida la solución q^* , el nodo de cinemática inversa debe publicar los valores articulares en:

`/joint_states`

mediante mensajes:

`sensor_msgs/msg/JointState`.

Esto permite que `robot_state_publisher` actualice la configuración del robot y que el resultado sea visible en RViz2.

INSTRUCCIÓN: Los nombres y el orden de las articulaciones publicados deben coincidir con los definidos por el modelo URDF/XACRO del robot.

VII-I. Verificación mediante cinemática directa

La solución obtenida por IK debe verificarse nuevamente mediante la cinemática directa:

$$p_f = p(q^*), \quad (26)$$

y comprobar:

$$\|p_d - p_f\|_2 < \varepsilon. \quad (27)$$

INSTRUCCIÓN: El nodo de FK debe permitir comprobar numéricamente la posición alcanzada. En la ejecución deben mostrarse, como mínimo, el objetivo solicitado, la posición final calculada y el error obtenido.

VIII. IMPLEMENTACIÓN EN PYTHON Y ROS 2

VIII-A. Herramientas

La implementación debe ser compatible con:

- Ubuntu 24.04;
- ROS 2 Jazzy;
- Python 3;
- RViz2;
- `robot_state_publisher`;
- `joint_state_publisher_gui`;
- NumPy y/o SymPy;
- SciPy, únicamente si el grupo la utiliza y la declara como dependencia.

VIII-B. Nodo de cinemática directa

INSTRUCCIÓN: Crear un nodo propio, por ejemplo `fk_node.py`, que se suscriba a `/joint_states` y calcule la transformación T_0^n utilizando las matrices DH desarrolladas por el grupo. No debe limitarse a consultar TF y presentar ese resultado como si fuera su cinemática directa. TF puede utilizarse únicamente como referencia de validación.

El nodo debe, como mínimo:

1. suscribirse a `/joint_states`;
2. leer y ordenar correctamente las articulaciones del robot;
3. evaluar las matrices DH;
4. calcular T_0^n ;
5. obtener la posición (x, y, z) ;
6. obtener la orientación del efector final, por ejemplo como cuaternión;
7. mostrar los resultados de forma clara en consola.

INSTRUCCIÓN: Publicar adicionalmente la pose como `PoseStamped` es válido, pero no es requisito obligatorio de este primer parcial. Sí es obligatorio calcular posición y orientación en FK.

VIII-C. Nodo de cinemática inversa

INSTRUCCIÓN: Crear un nodo propio, por ejemplo `ik_node.py`, que se suscriba obligatoriamente a `/target` con mensajes `geometry_msgs/msg/Point`, resuelva la IK de posición y publique la solución articular en `/joint_states`.

El flujo mínimo esperado es:

`/target (Point: x, y, z)`

```

      |
      v
    ik_node
      |
      v
  /joint_states
      |
      +--> robot_state_publisher --> RViz2
      |
      +--> fk_node --> verificacion numerica

```

El nodo de IK debe reportar:

- objetivo recibido (x_d, y_d, z_d);
- configuración inicial q_0 ;
- solución articular obtenida q^* ;
- número de iteraciones;
- posición alcanzada;
- error final;
- indicación clara de convergencia o no convergencia.

VIII-D. Interacción con Joint State Publisher GUI

INSTRUCCIÓN: El Joint State Publisher GUI también publica `/joint_states`. Para mover manualmente el robot y probar FK puede utilizarse normalmente. Para las pruebas en las que `ik_node.py` publique la solución en `/joint_states`, no deben mantenerse dos publishers de estados articulares controlando simultáneamente el robot. El grupo debe cerrar el GUI durante esa prueba o documentar una arquitectura equivalente que evite el conflicto.

IX. WORKSPACE ROS 2

IX-A. Estructura mínima

INSTRUCCIÓN: Mostrar la estructura real del workspace del grupo. No incluir `build/`, `install/` ni `log/` dentro del repositorio GitHub.

```

grupo_XX_robot_ws/
|-- src/
|   |-- grupoXX_robot_bringup/
|   |   |-- launch/
|   |   |   '-- display.launch.py
|   |   |-- package.xml
|   |   '-- setup.py / CMakeLists.txt
|   |-- grupoXX_robot_kinematics/
|   |   |-- grupoXX_robot_kinematics/
|   |   |   |-- fk_node.py
|   |   |   '-- ik_node.py
|   |   |-- resource/
|   |   |-- package.xml
|   |   |-- setup.py
|   |   '-- setup.cfg
|-- dependencias.repos
|-- instalar.sh
|-- entorno.sh
|-- README.md
|-- requirements.txt
'-- .gitignore

```

INSTRUCCIÓN: El paquete de descripción oficial del fabricante puede obtenerse mediante `dependencias.repos` o mediante el script de instalación. No es necesario duplicar repositorios externos completos dentro de GitHub. Es preferible declarar su URL y fijar una rama, tag o commit conocido.

IX-B. Compilación

Desde la raíz del workspace:

```

source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

rosdep install --from-paths src --ignore-src -r -y

colcon build --symlink-install
source install/setup.bash

```

IX-C. Ejecución del robot

```
ros2 launch grupoXX_robot_bringup display.launch.py
```

IX-D. Ejecución de los nodos propios

Terminal para cinemática directa:

```
ros2 run grupoXX_robot_kinematics fk_node
```

Terminal para cinemática inversa:

```
ros2 run grupoXX_robot_kinematics ik_node
```

En otra terminal se envía un objetivo:

```
ros2 topic pub /target geometry_msgs/msg/Point \
"x:_0.40,_y:_0.20,_z:_0.50" --once
```

INSTRUCCIÓN: Reemplazar los nombres anteriores por los nombres reales del paquete y ejecutables del grupo. Los valores del punto son de ejemplo. Todos los comandos del informe deben poder copiarse y ejecutarse.

X. VALIDACIÓN DE LA CINEMÁTICA DIRECTA

INSTRUCCIÓN: Realizar al menos tres casos de prueba dentro de los límites articulares. Debe existir al menos una configuración no trivial. Evitar validar únicamente con todos los ángulos en cero.

El error de posición se calculará como:

$$e_p = \|p_{\text{calc}} - p_{\text{ref}}\|_2. \quad (28)$$

INSTRUCCIÓN: Explicar qué se utiliza como referencia: TF del modelo, una medición obtenida del modelo oficial, o una verificación equivalente. RViz2 por sí solo es una visualización; la comparación numérica debe provenir de datos reproducibles.

X-A. Pruebas por transformaciones parciales

INSTRUCCIÓN: Se recomienda validar la cadena de manera incremental, especialmente cuando existe un error de frames. Por ejemplo: A_0^1 , $A_0^1 A_1^2$, $A_0^1 A_1^2 A_2^3$, etc. Incluir capturas o resultados de los eslabones relevantes.

Cuadro IV
VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Caso	q [rad]	p_{calc} [m]	p_{ROS} [m]	Error pos. [m]	Error ori. [rad]
1	[...]	[...]	[...]	[...]	[...]
2	[...]	[...]	[...]	[...]	[...]
3	[...]	[...]	[...]	[...]	[...]

Cuadro V
PRUEBAS DE CINEMÁTICA INVERSA

Caso	$p_d = [x_d, y_d, z_d]$	q_0	q^*	Iteraciones	Error final [m]
1	[...]	[...]	[...]	[...]	[...]
2	[...]	[...]	[...]	[...]	[...]
3	[...]	[...]	[...]	[...]	[...]

XI. VALIDACIÓN DE LA CINEMÁTICA INVERSA

INSTRUCCIÓN: Realizar al menos tres objetivos cartesianos alcanzables enviados mediante `/target`. Para cada caso mostrar (x_d, y_d, z_d) , configuración inicial, solución articular, posición final alcanzada, número de iteraciones y error final. Si el algoritmo no converge en un caso, documentarlo en vez de ocultarlo.

INSERTAR CAPTURAS DE VALIDACIÓN
DE LA CINEMÁTICA INVERSA

Figura 3. Validación de objetivos de cinemática inversa en RViz2.

XII. REPOSITORIO GITHUB Y REPRODUCIBILIDAD

INSTRUCCIÓN: Esta sección es obligatoria. El repositorio debe permitir que otra persona clone el proyecto y lo ejecute en una computadora limpia con Ubuntu 24.04 y ROS 2 Jazzy siguiendo únicamente el README. El enlace debe estar disponible antes de la entrega. Si el repositorio es privado, el docente debe tener acceso antes del plazo.

XII-A. Enlace

- Repositorio: `https://github.com/USUARIO/REPOSITORIO`
- Rama de entrega: `[main]`
- Commit evaluado: `[HASH DEL COMMIT FINAL]`

XII-B. Contenido mínimo del repositorio

El repositorio debe incluir:

1. README.md completo;
2. código fuente de los paquetes ROS 2 desarrollados por el grupo;
3. launch files propios;
4. `package.xml`, `setup.py`/`CMakeLists.txt` y archivos necesarios para compilar;

5. `dependencias.repos` o mecanismo equivalente para recuperar dependencias externas;
6. `install.sh` o instrucciones reproducibles de instalación;
7. `entorno.sh` si se requieren variables de entorno;
8. `requirements.txt` si se utilizan dependencias Python adicionales;
9. `.gitignore`;
10. carpeta `docs/` con el informe final en PDF;
11. capturas o datos estrictamente necesarios para reproducir las pruebas.

XII-C. Archivos que no deben subirse

No deben versionarse:

```
build/
install/
log/
__pycache__/
*.pyc
.vscode/
.idea/
```

INSTRUCCIÓN: No subir el workspace entero comprimido como sustituto del repositorio. GitHub debe contener los archivos fuente y la información necesaria para reconstruir el workspace.

XII-D. Requisitos del README

El README.md debe contener, como mínimo:

1. nombre del proyecto, robot y autores;
2. objetivo del trabajo;
3. software y versiones requeridas;
4. instrucciones de clonación;
5. instalación de dependencias;
6. recuperación del paquete oficial del robot;
7. compilación con `colcon build` `-symlink-install`;
8. comando `source install/setup.bash`;
9. comando exacto para abrir RViz2;
10. comando para ejecutar FK;
11. comando para ejecutar IK;
12. ejemplo concreto de una prueba;
13. tópicos, servicios o parámetros utilizados;

14. estructura del repositorio;
15. errores conocidos o consideraciones particulares.

XII-E. Procedimiento mínimo de reproducción

Otra persona debe poder ejecutar:

```
git clone https://github.com/USUARIO/REPOSITORIO.git
cd REPOSITORIO

chmod +x instalar.sh
./instalar.sh

source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source install/setup.bash

ros2 launch grupoXX_robot_bringup display.launch.py
```

y posteriormente:

```
ros2 run grupoXX_robot_kinematics fk_node
ros2 run grupoXX_robot_kinematics ik_node
```

INSTRUCCIÓN: Si estos comandos no son suficientes para reproducir el proyecto, el README está incompleto. Todo paso manual adicional debe documentarse.

XII-F. Dependencias externas reproducibles

INSTRUCCIÓN: Las dependencias externas deben quedar fijadas a una rama estable, tag o commit conocido. No depender únicamente de “la última versión” de un repositorio, porque una actualización futura puede romper el proyecto.

Ejemplo conceptual de `dependencias.repos`:

```
repositories:
  robot_description:
    type: git
    url: https://github.com/ORGANIZACION/REPOSITORIO.git
    version: RAMA_TAG_O_COMMIT
```

Posteriormente:

```
mkdir -p src
vcs import src < dependencias.repos
```

XIII. RESULTADOS Y DISCUSIÓN

INSTRUCCIÓN: Comparar los resultados matemáticos con ROS 2. Analizar errores, dificultades de asignación de frames, sensibilidad de la IK a la semilla inicial, comportamiento cerca de singularidades y cualquier diferencia entre el modelo matemático y el URDF/XACRO. Evitar una sección puramente descriptiva.

[DESARROLLO DEL GRUPO.]

XIV. CONCLUSIONES

INSTRUCCIÓN: Redactar entre tres y cinco conclusiones técnicas. Deben responder qué se logró, qué se validó, qué limitaciones se encontraron y qué aprendió el grupo sobre DH, FK, Jacobiano, IK y ROS 2. No repetir literalmente el resumen.

- [Conclusión 1.]
- [Conclusión 2.]
- [Conclusión 3.]

XV. REFERENCIAS

INSTRUCCIÓN: Incluir documentación oficial del fabricante, repositorio oficial del modelo ROS 2 y bibliografía utilizada para DH, FK, Jacobiano e IK. Utilizar un formato consistente.

REFERENCIAS

- [1] KUKA, “KR AGILUS – KR 7 R900-3, documentación técnica del fabricante,” KUKA Robotics, consultado el 4 de octubre de 2026.
- [2] [Organización], “[Repositorio ROS 2 oficial o utilizado],” GitHub, [URL y commit/tag utilizado].
- [3] [Autor], [Título del libro o recurso de robótica], [Editorial/año].

APÉNDICE A

CHECKLIST PREVIO A LA ENTREGA

- ☐ Todos los autores y el robot están correctamente identificados.
- ☐ Los frames aparecen en una figura propia.
- ☐ La tabla DH coincide con las matrices utilizadas.
- ☐ Se muestran todas las transformaciones necesarias.
- ☐ Se incluyen $p(q)$ y $R(q)$.
- ☐ Se presenta y explica el Jacobiano posicional.
- ☐ El nodo IK recibe el objetivo mediante `/target (Point)`.
- ☐ El nodo IK publica la solución en `/joint_states`.
- ☐ Se documentan q_0 , tolerancia, límites e iteraciones.
- ☐ Hay al menos tres pruebas de FK.
- ☐ Hay al menos tres pruebas de IK.
- ☐ Los resultados numéricos se comparan con ROS 2.
- ☐ El repositorio GitHub es accesible.
- ☐ El README permite reproducir el proyecto desde cero.
- ☐ `build/`, `install/` y `log/` no están en GitHub.
- ☐ El commit final está indicado en el informe.
- ☐ Se eliminaron todos los textos [PENDIENTES] e instrucciones no requeridas.